

Cours Segmentation d'images

et données rares

Dr. Khaoula Ben Ali

Oct 3, 2026

1. Introduction à la segmentation d'images

La segmentation d'images consiste à attribuer une classe à chaque pixel d'une image. Là où la classification dit « cette image contient une pizza » et la détection dessine un rectangle autour, la segmentation indique exactement quels pixels appartiennent à la pizza.

Le résultat est un masque : une image de même taille que l'entrée, où chaque pixel porte le numéro de sa classe. Pour une image de $H \times W$ pixels et C classes, le modèle produit un tenseur de taille $C \times H \times W$, puis on garde pour chaque pixel la classe la plus probable.

Les trois types de segmentation

Type	Question posée	Exemple
Sémantique	À quelle classe appartient ce pixel ?	Tous les pixels « tomate » ont la même couleur
D'instance	À quel objet précis appartient ce pixel ?	Chaque tomate a sa propre couleur
Panoptique	Les deux à la fois	Objets comptables séparés + fond (« stuff ») segmenté

Applications

- Médecine : délimiter une tumeur, un organe ou une lésion sur une IRM ou une échographie.
- Alimentation et nutrition : reconnaître les aliments d'une assiette pour estimer les portions et les calories (jeu de données FoodSeg103).
- Véhicules autonomes : séparer la route, les piétons et les véhicules.
- Agriculture et environnement : cartographier les cultures ou les sols sur des images satellites.
- Industrie : détecter des défauts sur une pièce ou un produit.

La segmentation est plus difficile que la classification : le modèle doit à la fois comprendre le contenu (quoi) et localiser précisément les contours (où).

2. La rareté des données annotées (data scarcity)

Le principal frein à la segmentation n'est pas le modèle, mais le manque de données annotées. Un réseau profond a des millions de paramètres et a besoin de nombreux exemples pour bien généraliser.

Pourquoi les annotations sont rares

- L'annotation est très coûteuse. Il faut dessiner le contour de chaque objet, pixel par pixel. Une seule image peut demander de quelques minutes à plus d'une heure.
- L'expertise est parfois nécessaire. En médecine, seul un radiologue peut délimiter une tumeur de façon fiable.

- Certaines classes sont rares. Dans un jeu d'aliments, le riz ou le pain apparaissent souvent, mais certains fruits ou plats régionaux très peu.
- Les annotations sont ambiguës. Deux annotateurs ne tracent jamais exactement le même contour, surtout sur des aliments mélangés.

Les conséquences

1. Le surapprentissage (overfitting) : le modèle apprend les images d'entraînement par cœur et échoue sur de nouvelles images. On le voit quand la perte d'entraînement baisse alors que la perte de validation stagne ou remonte.
2. Le déséquilibre des classes : le fond et les classes fréquentes dominent les pixels. Le modèle les prédit bien, mais ignore les classes rares.
3. Une faible généralisation : un modèle entraîné sur des photos d'un seul pays ou d'un seul appareil marche mal sur d'autres conditions d'éclairage ou de cadrage.

Les grandes familles de solutions

Solution	Principe	Section
Augmentation de données	Créer de nouvelles images à partir des existantes	3
Données synthétiques	Générer des images et leurs masques avec un modèle génératif	3
Pseudo-étiquetage	Annoter automatiquement des images non étiquetées	3
Apprentissage par transfert	Partir d'un encodeur pré-entraîné sur un grand jeu (ImageNet)	4
Fonctions de perte adaptées	Donner plus de poids aux classes rares (Dice, focal)	5

Ces solutions se combinent. Pour savoir laquelle apporte vraiment un gain, on utilise une étude d'ablation (section 6).

3. Les techniques d'augmentation de données

L'augmentation crée de nouvelles variantes des images d'entraînement, pour que le modèle voie plus de situations sans annotation supplémentaire. Règle d'or en segmentation : toute transformation géométrique appliquée à l'image doit être appliquée de la même façon au masque, sinon les pixels ne correspondent plus à leur classe.

3.1 Augmentations classiques

Famille	Exemples	Appliquée au masque ?
Géométrique	Retournement horizontal, rotation, recadrage aléatoire, mise à l'échelle	Oui, la même transformation
Photométrique	Luminosité, contraste, saturation, teinte	Non, le masque ne change pas
Bruit et flou	Bruit gaussien, flou, compression JPEG	Non
Déformation	Déformation élastique, distorsion de grille	Oui

La bibliothèque Albumentations applique automatiquement la bonne transformation à l'image et au masque. Ces augmentations se font à la volée : à chaque epoch, le modèle voit une version différente de chaque image.

Attention : les augmentations s'appliquent seulement aux données d'entraînement. Les données de validation et de test restent intactes, pour mesurer la vraie performance.

3.2 Augmentations avancées par mélange

- CutMix : on colle un rectangle d'une image dans une autre, avec la partie correspondante de son masque.
- Copy-Paste : on découpe un objet selon son masque et on le colle dans une autre image. Très efficace pour les classes rares : on peut multiplier les apparitions d'un aliment peu fréquent.
- Mosaic : on assemble quatre images en une seule, ce qui crée de nouveaux contextes.

3.3 Données synthétiques génératives

Des modèles génératifs créent des images entièrement nouvelles :

- Les GAN (réseaux antagonistes génératifs) : un générateur produit des images, un discriminateur juge si elles sont réalistes.
- Les modèles de diffusion (Stable Diffusion, par exemple) : ils génèrent des images à partir d'un texte (« une assiette de riz avec des légumes »).

Le problème : une image générée n'a pas de masque. Il faut donc l'annoter automatiquement.

3.4 Pseudo-étiquetage avec des modèles de fondation

Des modèles pré-entraînés sur d'énormes jeux de données permettent d'annoter sans intervention humaine :

1. SAM (Segment Anything Model) découpe automatiquement les objets d'une image, mais sans savoir ce qu'ils sont.
2. CLIP relie une image à un texte : il permet de donner un nom (une classe) à chaque région découpée par SAM.
3. On filtre les résultats peu fiables, par exemple en gardant seulement les images où les étiquettes correspondent à la classe attendue.

C'est la démarche du notebook FoodSeg103 étudié : des images synthétiques sont étiquetées par SAM + CLIP, puis ajoutées au jeu d'entraînement.

3.5 Risques des données synthétiques

- Un écart de domaine : les images générées ne ressemblent pas exactement aux vraies photos. Le modèle peut apprendre des détails artificiels.
- Des masques bruités : une erreur de SAM ou de CLIP devient une erreur d'apprentissage.
- Une fuite d'information : les données synthétiques ne doivent jamais être générées à partir des images de test, ni ajoutées au jeu de test.
- Une évaluation trompeuse : il faut toujours mesurer la performance sur des images réelles.

3.6 L'apprentissage par transfert

Plutôt que d'entraîner un réseau à partir de zéro, on part d'un encodeur pré-entraîné sur ImageNet (plus d'un million d'images). Il sait déjà reconnaître des contours, des textures et des formes. Une stratégie courante se déroule en deux phases :

1. Geler l'encodeur et entraîner seulement le décodeur pendant quelques epochs ;
2. Dégeler l'encodeur et affiner tout le réseau avec un petit taux d'apprentissage.

Cela protège les connaissances pré-acquises au début de l'entraînement, quand le décodeur produit encore des gradients désordonnés.

4. Les modèles neuronaux de segmentation

Presque tous les modèles de segmentation suivent une architecture encodeur–décodeur :

- L'encodeur réduit progressivement la taille de l'image et extrait des caractéristiques de plus en plus abstraites (contours, textures, puis objets). C'est souvent un réseau de classification pré-entraîné : ResNet, EfficientNet, MiT.
- Le décodeur remonte à la résolution d'origine pour produire une prédiction par pixel.

La difficulté centrale : l'encodeur perd les détails spatiaux en réduisant l'image, alors que la segmentation a besoin de contours précis. Chaque architecture répond différemment à ce problème.

4.1 FCN (Fully Convolutional Network, 2015)

Le premier réseau entièrement convolutif pour la segmentation. Il remplace les couches entièrement connectées d'un réseau de classification par des convolutions, puis agrandit la carte de sortie par suréchantillonnage. Ses contours restent grossiers.

4.2 U-Net (2015)

Conçu à l'origine pour l'imagerie biomédicale, avec peu de données. Sa forme en « U » associe un encodeur et un décodeur symétriques, reliés par des connexions de saut (skip connections) : à chaque niveau, les cartes détaillées de l'encodeur sont transmises directement au décodeur. Le décodeur récupère ainsi les détails perdus, ce qui donne des contours fins.

C'est le modèle du notebook FoodSeg103, avec un encodeur ResNet-101 pré-entraîné sur ImageNet.

4.3 DeepLabv3 / DeepLabv3+ (2017–2018)

DeepLab utilise des convolutions dilatées (trous) : des filtres « troués » qui voient une zone plus large sans réduire l'image. Son module ASPP (Atrous Spatial Pyramid Pooling) applique plusieurs dilations en parallèle pour capter les objets à différentes échelles. DeepLabv3+ ajoute un petit décodeur pour affiner les contours.

4.4 SegFormer (2021)

SegFormer remplace l'encodeur convolutif par un Transformer hiérarchique. Le mécanisme d'attention permet à chaque région de l'image de « regarder » toutes les autres, ce qui capte bien le contexte global. Son décodeur est très léger (simples couches MLP). Il est performant, mais demande en général plus de données et de mémoire.

4.5 Comparaison

Modèle	Idée clé	Point fort	Point faible
FCN	Convolutions seules + suréchantillonnage	Simple, historique	Contours grossiers
U-Net	Connexions de saut encodeur → décodeur	Contours précis, efficace avec peu de données	Contexte global limité
DeepLabv3+	Convolutions dilatées + ASPP	Objets de tailles variées	Plus lourd en calcul
SegFormer	Encodeur Transformer + décodeur MLP	Contexte global, état de l'art	Plus gourmand en données et en mémoire

Avec peu de données, un U-Net ou un DeepLab avec un encodeur pré-entraîné est souvent un bon point de départ. Les Transformers donnent leurs meilleurs résultats quand les données sont plus abondantes, ou quand ils ont été pré-entraînés sur de très grands jeux.

5. Fonctions de perte et métriques d'évaluation

La fonction de perte guide l'apprentissage : elle mesure l'erreur que le modèle cherche à réduire. Les métriques mesurent la qualité finale. Les deux ne sont pas toujours les mêmes.

5.1 Fonctions de perte

Entropie croisée (Cross-Entropy)

Elle pénalise chaque pixel mal classé, indépendamment des autres. C'est la perte standard, mais elle est dominée par les classes qui occupent beaucoup de pixels, comme le fond.

$$L_{CE} = -(1/N) \times \sum_i \sum_c y_{i,c} \times \log(p_{i,c})$$

Dice loss

Elle mesure le recouvrement entre la zone prédite et la zone réelle de chaque classe. Une petite classe compte autant qu'une grande, ce qui aide face au déséquilibre.

$$L_{Dice} = 1 - (2 \times \sum(p \cdot g) + \epsilon) / (\sum p + \sum g + \epsilon)$$

Perte combinée

On additionne souvent les deux, par exemple $0,7 \times \text{Cross-Entropy} + 0,3 \times \text{Dice}$, comme dans le notebook FoodSeg103. L'entropie croisée stabilise l'apprentissage pixel par pixel, et la Dice améliore le recouvrement des régions.

$$L = 0.7 \times L_{CE} + 0.3 \times L_{Dice}$$

Focal loss

Variante de l'entropie croisée qui réduit le poids des pixels faciles, pour concentrer l'apprentissage sur les pixels difficiles.

5.2 Métriques

Métrique	Formule	Ce qu'elle mesure
Exactitude par pixel	Pixels bien classés / total des pixels	Proportion de pixels corrects (trompeuse si le fond domine)
IoU	Intersection / Union	Recouvrement prédiction / vérité (sévère pour petits objets)
mIoU	Moyenne des IoU par classe	Métrique de référence en segmentation
Dice (F1 des pixels)	$2 \times \text{Intersection} / (\text{Prédits} + \text{Réels})$	Recouvrement, plus indulgent que l'IoU

Exemple du piège de l'exactitude : si le fond couvre 80 % d'une image, un modèle qui prédit « fond » partout obtient 80 % d'exactitude, sans reconnaître un seul aliment. Son mIoU, lui, sera très faible. C'est pourquoi on rapporte toujours le mIoU en segmentation.

Relation entre Dice et IoU :

$$Dice = 2 \times IoU / (1 + IoU)$$

Les deux sont liés : un modèle meilleur en IoU est aussi meilleur en Dice.

5.3 Bonnes pratiques d'évaluation

- Trois jeux distincts : entraînement (apprendre), validation (choisir les hyperparamètres et le meilleur modèle), test (mesurer la performance finale une seule fois).
- Calculer les métriques sur toute l'epoch, en cumulant les intersections et les unions, plutôt qu'en faisant la moyenne de chaque batch.
- Rapporter les résultats par classe, pour voir si les classes rares sont bien reconnues.

6. L'étude d'ablation

Une étude d'ablation consiste à retirer (ou modifier) un seul élément à la fois d'un système, pour mesurer sa contribution réelle. Le mot vient de la médecine, où l'on retire une partie d'un organe pour comprendre son rôle.

Elle répond à la question : « Ce composant améliore-t-il vraiment le résultat, ou le gain vient-il d'ailleurs ? »

6.1 Pourquoi c'est indispensable

Un modèle final combine souvent plusieurs choix : un encodeur pré-entraîné, des augmentations, des données synthétiques, une perte combinée, un gel de l'encodeur. Si le modèle final est meilleur, on ne sait pas lequel de ces choix explique le gain. Sans ablation, on ne peut pas affirmer, par exemple, que les données synthétiques ont aidé.

6.2 La méthode

1. Définir une référence (baseline) : la configuration la plus simple, ou le modèle complet.
2. Changer un seul facteur à la fois, en gardant tout le reste identique : même architecture, mêmes hyperparamètres, même nombre d'epochs, mêmes données de validation.
3. Mesurer avec la même métrique (mIoU de préférence), sur le même jeu de test réel.
4. Répéter chaque expérience avec plusieurs graines aléatoires (par exemple 3), et rapporter la moyenne et l'écart-type. Un gain plus petit que l'écart-type n'est pas significatif.
5. Présenter les résultats dans un tableau, avec l'écart par rapport à la référence.

6.3 Exemple : le notebook FoodSeg103

Le flag `USE_AUGMENTED_DATA` du notebook est déjà une ablation : on compare un entraînement sans et avec images synthétiques, toutes choses égales par ailleurs. On peut étendre l'étude ainsi (les valeurs sont à remplir avec les résultats obtenus) :

Expérience	Données synth.	Aug. classiques	Perte	Gel encodeur	mIoU (moy ± SD)
A – Baseline	Non	Oui	CE+Dice	Oui	[]
B – + Données synth.	Oui	Oui	CE+Dice	Oui	[]
C – Sans augmentations	Non	Non	CE+Dice	Oui	[]
D – Cross-Entropy seule	Non	Oui	CE	Oui	[]
E – Sans gel encodeur	Non	Oui	CE+Dice	Non	[]
F – Encodeur non pré-entraîné	Non	Oui	CE+Dice	Oui	[]

Chaque ligne ne change qu'un seul élément par rapport à la ligne A. Si B donne un mIoU plus élevé que A, au-delà de l'écart-type, on peut conclure que les données synthétiques aident.

6.4 Analyses complémentaires utiles

- Effet de la quantité : entraîner avec 25 %, 50 %, 75 % et 100 % des données réelles, avec et sans données synthétiques. On voit si le synthétique aide surtout quand les données réelles sont rares.
- Effet par classe : comparer l'IoU de chaque classe. Les données synthétiques aident souvent davantage les classes rares.
- Ratio réel / synthétique : tester plusieurs proportions (par exemple 1:0,5, 1:1, 1:2). Trop de synthétique peut dégrader les résultats.

6.5 Erreurs fréquentes

- Changer plusieurs facteurs à la fois : on ne sait plus lequel explique le résultat.
- Comparer des expériences avec un nombre d'epochs différent : un modèle entraîné sur plus d'images voit plus de mises à jour par epoch.
- Évaluer sur le jeu de validation utilisé pour choisir le meilleur modèle : les résultats sont optimistes.
- Conclure à partir d'une seule exécution, alors que la variabilité entre graines peut dépasser le gain observé.

7. Points clés et questions de révision

À retenir

1. La segmentation classe chaque pixel ; elle exige à la fois de comprendre le contenu et de localiser les contours.
2. Le manque de données annotées est le principal obstacle, car l'annotation pixel par pixel est longue et coûteuse.
3. L'augmentation classique doit transformer l'image et le masque de la même façon, et s'applique seulement à l'entraînement.
4. Les données synthétiques (diffusion + SAM + CLIP) peuvent aider, mais leur effet doit être prouvé sur des images réelles.
5. U-Net, grâce à ses connexions de saut, reste un excellent choix quand les données sont rares ; SegFormer exploite mieux le contexte global.
6. Le mIoU est la métrique de référence ; l'exactitude par pixel est trompeuse quand le fond domine.
7. Une étude d'ablation change un seul facteur à la fois, avec plusieurs graines, pour mesurer la vraie contribution de chaque choix.

Questions de révision

1. Quelle est la différence entre segmentation sémantique, d'instance et panoptique ?
2. Pourquoi une rotation doit-elle être appliquée au masque, alors qu'un changement de luminosité ne l'est pas ?
3. Quel rôle jouent les connexions de saut dans U-Net ?
4. Pourquoi combine-t-on souvent la Cross-Entropy et la Dice loss ?
5. Un modèle atteint 85 % d'exactitude par pixel mais un mIoU de 20 %. Que peut-on en conclure ?
6. Quels sont les risques d'ajouter des images synthétiques au jeu d'entraînement ?
7. Vous ajoutez des données synthétiques et changez la fonction de perte dans la même expérience, et le mIoU augmente de 2 points. Pourquoi ne pouvez-vous pas conclure que les données synthétiques ont aidé ?
8. Pourquoi faut-il répéter une expérience avec plusieurs graines aléatoires ?
9. Pourquoi geler l'encodeur pré-entraîné au début de l'entraînement ?
10. Proposez un plan d'ablation pour évaluer l'apport d'un nouvel encodeur Transformer par rapport à ResNet-101.